

3.4 Builder Pattern

Builder Design Pattern เป็นแพตเทิร์นเพื่อสร้างอ็อบเจกต์ที่มีความซับซ้อน และมีหลายส่วนประกอบ โดยแยกกระบวนการสร้างอ็อบเจกต์ออกจากตัวโครงสร้างหลัก ช่วยให้สามารถสร้างอ็อบเจกต์ที่ซับซ้อนได้ทีละขั้นตอน โดยไม่จำเป็นต้องใช้ Constructor ที่มีพารามิเตอร์จำนวนมาก หรือมีปัญหาในการจัดการกับพารามิเตอร์ที่ไม่จำเป็น ทำให้สามารถสร้างอ็อบเจกต์หลายแบบจากโครงสร้างเดียวกันได้

:: *Builder Pattern* ::

สร้างอ็อบเจกต์ที่มีความซับซ้อนและมีหลายส่วนประกอบ

สร้างอ็อบเจกต์ที่ซับซ้อนได้ทีละขั้นตอน โดยไม่จำเป็นต้องใช้ Constructor ที่มีพารามิเตอร์จำนวนมาก

ตัวอย่างปัญหา

การสร้างบ้านมีรายละเอียดมากและซับซ้อน เช่น บางบ้านมีสระว่ายน้ำ บางบ้านไม่มี บางบ้านมีโรงรถ Builder Pattern เปรียบเหมือนการคุยกับวิศวกรโยธาเพื่อระบุสเปกทีละอย่าง (Step-by-step) จนครบ แล้วค่อยสั่งให้ประกอบเป็นบ้าน 1 หลัง ช่วยแก้ปัญหการส่ง Parameter จำนวนมากไปให้ Constructor พร้อมกัน

ตัวอย่าง Main program

```
// เคสที่ 1: ลูกค้าอยากได้บ้านหรู
HouseBuilder luxuryContractor = new LuxuryHouseBuilder();
CivilEngineer engineerA = new
    CivilEngineer(luxuryContractor);
engineerA.constructHouse();
House luxuryHouse = luxuryContractor.getHouse();
luxuryHouse.showHouse();

// เคสที่ 2: ลูกค้าอยากได้บ้านมินิมอล
HouseBuilder simpleContractor = new SimpleHouseBuilder();
CivilEngineer engineerB = new
    CivilEngineer(simpleContractor);
engineerB.constructHouse();
House simpleHouse = simpleContractor.getHouse();
simpleHouse.showHouse();
```

3.4.1 วัตถุประสงค์และตัวอย่าง

วัตถุประสงค์

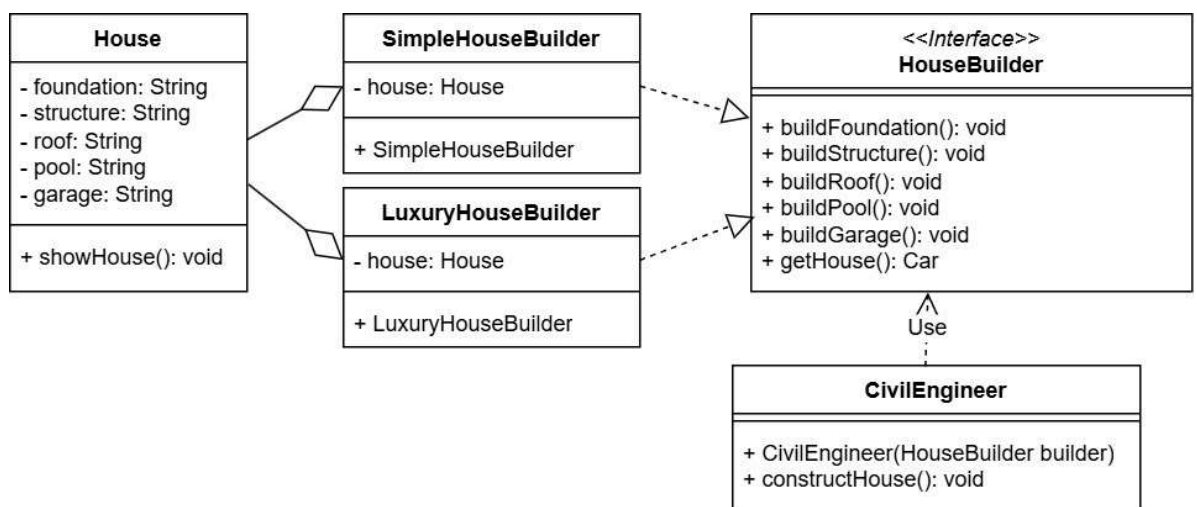
- สร้างอ็อบเจกต์ที่ซับซ้อน หากอ็อบเจกต์มีหลายส่วนประกอบ แต่ละส่วนมีความสัมพันธ์กัน
- ควบคุมการสร้างทีละขั้นตอน ช่วยให้กระบวนการสร้างมีความยืดหยุ่น เช่น เลือกเฉพาะบางส่วนที่ต้องการ
- ลดความซับซ้อนในโค้ดหลัก ช่วยให้การสร้างอ็อบเจกต์ไม่ต้องกำหนดค่าทั้งหมดใน Constructor เดียว
- รองรับการขยายระบบ เพิ่มความสามารถในการสร้างอ็อบเจกต์แบบใหม่ได้ง่าย

ตัวอย่างการใช้งาน

การสร้างบ้านมีรายละเอียดมากและซับซ้อน เช่น บางบ้านมีสระว่ายน้ำ บางบ้านไม่มี บางบ้านมีโรงรถ Builder Pattern เปรียบเหมือนการคุยกับวิศวกรโยธาเพื่อระบุสเปกทีละอย่าง (Step-by-step) จนครบ แล้วค่อยสั่งให้ประกอบเป็นบ้าน 1 หลัง ช่วยแก้ปัญหการส่ง Parameter จำนวนมากไปให้ Constructor พร้อมกัน

ส่วนประกอบของระบบ มีดังนี้

- Product (House) บ้านที่จะถูกสร้างออกมา
- Builder Interface (HouseBuilder) กำหนดขั้นตอนการสร้างบ้าน
- Concrete Builder (LuxuryHouseBuilder, SimpleHouseBuilder) บ้านแต่ละแบบ
- Director (CivilEngineer) วิศวกรโยธาผู้ควบคุมลำดับขั้นตอนการสร้าง



ภาพที่ xx บ้านมีส่วนประกอบที่ซับซ้อน โดยใช้ Builder pattern

Product class ได้แก่ House คลาสของออบเจกต์ที่ต้องการสร้าง House มีรายละเอียดต่าง ๆ ได้แก่ foundation, structure, roof, pool, garage มี Setter methods เพื่อให้ Builder สามารถกำหนดค่าให้กับออบเจกต์ได้

```
1. // 1. คลาสบ้านที่มีรายละเอียดมากและซับซ้อน
2. class House {
3.
4.     private String foundation;
5.     private String structure;
6.     private String roof;
7.     private String pool;
8.     private String garage;
9.
10.    public void setFoundation(String foundation) {
11.        this.foundation = foundation;
12.    }
13.    public void setStructure(String structure) {
14.        this.structure = structure;
15.    }
16.    public void setRoof(String roof) {
17.        this.roof = roof;
18.    }
19.    public void setPool(String pool) {
20.        this.pool = pool;
21.    }
22.    public void setGarage(String garage) {
23.        this.garage = garage;
24.    }
25.
26.    public void showHouse() {
27.        System.out.println("--- รายละเอียดบ้าน ---");
28.        System.out.println("ฐานราก: " + foundation);
29.        System.out.println("โครงสร้าง: " + structure);
30.        System.out.println("หลังคา: " + roof);
31.        System.out.println("สระว่ายน้ำ: " + pool);
32.        System.out.println("โรงรถ: " + garage);
33.    }
34.}
```

Builder Interface ได้แก่ HouseBuilder กำหนดอินเทอร์เฟซสำหรับ Builder ทั้งหมด ประกอบด้วยเมธอดสำหรับแต่ละขั้นตอนของการสร้าง ได้แก่ buildFoundation(), buildStructure(), buildRoof(), buildPool(), buildGarage มีเมธอด getHouse() สำหรับการดึงออบเจกต์ Product ที่สร้างเสร็จแล้ว

```

1. // 2. Builder Interface - กำหนดขั้นตอนมาตรฐานการสร้างบ้าน
2. interface HouseBuilder {
3.     void buildFoundation();
4.     void buildStructure();
5.     void buildRoof();
6.     void buildPool();
7.     void buildGarage();
8.     House getHouse();
9. }

```

Concrete Builder ได้แก่ SportsCarBuilder, SUVCarBuilder คลาสเหล่านี้เป็นคลาสที่ implement CarBuilder interface แต่ละ Concrete Builder มีหน้าที่รับผิดชอบสร้าง Product ในรูปแบบที่เฉพาะเจาะจง เช่น SimpleHouseBuilder, LuxuryHouseBuilder

```

1. class SimpleHouseBuilder implements HouseBuilder {
2.
3.     private House house = new House();
4.
5.     public void buildFoundation() {
6.         house.setFoundation("ฐานรากแผ่คอนกรีตมาตรฐาน"); }
7.     public void buildStructure() {
8.         house.setStructure("โครงสร้างคอนกรีตสำเร็จรูป (Precast)"); }
9.     public void buildRoof() {
10.        house.setRoof("หลังคาเมทัลชีททนความร้อน PU"); }
11.    public void buildGarage() {
12.        house.setGarage("โรงจอดรถขนาดสำหรับรถครอบครัว"); }
13.    public void buildPool() {
14.        house.setPool("ไม่มี"); }
15.    public House getHouse() { return this.house; }
16.}

```

```

1. class LuxuryHouseBuilder implements HouseBuilder {
2.
3.     private House house = new House();
4.
5.     public void buildFoundation() {
6.         house.setFoundation("เสาเข็มเจาะดอกลึกพิเศษคอนกรีตเสริมเหล็ก"); }
7.     public void buildStructure() {
8.         house.setStructure("โครงสร้างเหล็กกล้าและผนังกระจกนิรภัย"); }
9.     public void buildRoof() {
10.        house.setRoof("หลังคากระเบื้องเซรามิกสะท้อนความร้อน"); }
11.    public void buildPool() {
12.        house.setPool("สระว่ายน้ำระบบเกลือ"); }
13.    public void buildGarage() {
14.        house.setGarage("โรงรถ จอดได้ 4 คัน"); }
15.    public House getHouse() { return this.house; }
16.}

```

Director class ได้แก่ `CivilEngineer` ทำหน้าที่กำหนดชุดขั้นตอนการสร้างที่ซับซ้อน และใช้ซ้ำได้ `Director` ทำงานร่วมกับ `Builder` โดยการรับ `Builder` เข้ามาเป็นพารามิเตอร์ และเรียกเมธอด `build()` ของ `Builder` ตามลำดับที่กำหนดไว้ `Director` ไม่ทราบรายละเอียดของการสร้าง `Product` โดยตรง แต่จะใช้ `Builder` เพื่อดำเนินการสร้าง

```
1. // 4. Director - วิศวกรโยธาผู้ควบคุมลำดับขั้นตอนสร้างบ้าน
2. class CivilEngineer {
3.
4.     private HouseBuilder builder;
5.
6.     // รับ Builder เข้ามาผ่าน Interface (Polymorphism)
7.     public CivilEngineer(HouseBuilder builder) {
8.         this.builder = builder;
9.     }
10.
11.    // กำหนดขั้นตอนการสร้างที่ตายตัว เช่น ต้องทำฐานรากก่อนหลังคา
12.    public void constructHouse() {
13.        builder.buildFoundation();
14.        builder.buildStructure();
15.        builder.buildRoof();
16.        builder.buildPool();
17.        builder.buildGarage();
18.    }
19.}
```

Client Code `BuilderPatternDemo` หรือ main program คือส่วนที่ใช้งาน `Builder Pattern` โดยสร้าง `Director` และ `Concrete Builder` ที่ต้องการ

```
1. // 5. Client Code - การใช้งาน
2. public class BuilderPatternDemo {
3.     public static void main(String[] args) {
4.
5.         // เคสที่ 1: ลูกค้าอยากได้บ้านหรู
6.         HouseBuilder luxuryContractor = new LuxuryHouseBuilder();
7.         CivilEngineer engineerA = new
8.             CivilEngineer(luxuryContractor);
9.         engineerA.constructHouse();
10.        House luxuryHouse = luxuryContractor.getHouse();
11.        luxuryHouse.showHouse();
12.
13.        // เคสที่ 2: ลูกค้าอยากได้บ้านมินิมอล
14.        HouseBuilder simpleContractor = new SimpleHouseBuilder();
15.        CivilEngineer engineerB = new
16.            CivilEngineer(simpleContractor);
17.        engineerB.constructHouse();
18.        House simpleHouse = simpleContractor.getHouse();
19.        simpleHouse.showHouse();
20.    }
21.}
```

ผลลัพธ์ของโปรแกรม

--- รายละเอียดบ้าน ---

ฐานราก: เสาเข็มเจาะตอกลึกพิเศษคอนกรีตเสริมเหล็ก

โครงสร้าง: โครงสร้างเหล็กกล้าและผนังกระจกนิรภัย

หลังคา: หลังคากระเบื้องเซรามิกสะท้อนความร้อน

สระว่ายน้ำ: สระว่ายน้ำระบบเกลือ

โรงรถ: โรงรถ จอดได้ 4 คัน

--- รายละเอียดบ้าน ---

ฐานราก: ฐานรากแผ่คอนกรีตมาตรฐาน

โครงสร้าง: โครงสร้างคอนกรีตสำเร็จรูป (Precast)

หลังคา: หลังคาเมทัลชีทบุฉนวนกันความร้อน PU

สระว่ายน้ำ: ไม่มี

โรงรถ: โรงจอดรถขนาดสำหรับรถครอบครัว

ประโยชน์ของ Builder Pattern

- แยกความรับผิดชอบด้วยการสร้างออบเจกต์ที่ซับซ้อนถูกแยกออกจากคลาส Product และถูกจัดการโดย Builder
- มีความยืดหยุ่น สามารถสร้าง Product ในรูปแบบต่างๆ ได้โดยใช้ Builder ที่แตกต่างกัน
- โค้ดที่อ่านง่ายขึ้น แทนที่จะมี Constructor ที่มีพารามิเตอร์จำนวนมาก ทำให้โค้ดอ่านและบำรุงรักษาง่ายขึ้น
- ความสามารถในการควบคุมขั้นตอนการสร้าง สามารถกำหนดลำดับและเงื่อนไขในการสร้างส่วนประกอบต่าง ๆ ได้
- ลดจำนวน Constructor Overloading เนื่องจากไม่จำเป็นต้องสร้าง Constructor หลายตัวเพื่อรองรับการสร้างออบเจกต์ในรูปแบบที่ต่างกัน

3.4.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Builder Pattern เหมาะสมกับสถานการณ์ที่ต้องการสร้างออบเจกต์ที่ซับซ้อน หรือออบเจกต์ที่มีการกำหนดค่าหลายขั้นตอน ซึ่งในงานจริงมีหลายปัญหาที่สามารถแก้ไขได้ด้วย Builder Pattern ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 1 การสร้างรายงาน (Report Generator) ระบบต้องสร้างรายงานที่มีส่วนประกอบหลายอย่าง เช่น ชื่อรายงาน ตารางข้อมูล กราฟ และส่วนสรุป รายงานแต่ละแบบอาจมีองค์ประกอบต่างกัน การสร้างอ็อบเจกต์รายงานโดยใช้ Constructor ธรรมดาที่รองรับพารามิเตอร์หลายตัวจะทำให้โค้ดอ่านยากและดูซับซ้อน สามารถใช้ Builder Pattern เพื่อแยกกระบวนการสร้างรายงานออกเป็นขั้นตอน และสามารถกำหนดส่วนประกอบเฉพาะที่ต้องการ

Product (Report) คลาส Report คือตัวแทนของรายงานที่เราต้องการสร้าง ซึ่งประกอบด้วย title และ sections ที่เป็น List<String> เพื่อเก็บส่วนประกอบย่อย เช่น ตาราง กราฟ หรือสรุป มีเมธอด setTitle() และ addSection() สำหรับเพิ่มเนื้อหาเข้าไปในรายงานที่ละส่วน เมธอด displayReport() ใช้สำหรับแสดงผลลัพธ์ของรายงาน

```
1. // 1. Product: Report (อ็อบเจกต์รายงานที่ต้องการสร้าง)
2. class Report {
3.
4.     private String title;
5.
6.     // ใช้ List เพื่อเก็บส่วนประกอบของรายงาน
7.     private List<String> sections;
8.
9.     public Report() {
10.         this.sections = new ArrayList<>();
11.     }
12.
13.     public void setTitle(String title) {
14.         this.title = title;
15.     }
16.
17.     public void addSection(String sectionContent) {
18.         this.sections.add(sectionContent);
19.     }
20.
21.     public void displayReport() {
22.         System.out.println("--- " + title + " ---");
23.         for (String section : sections) {
24.             System.out.println(section);
25.         }
26.         System.out.println("-----\n");
27.     }
28. }
```

Builder Interface (ReportBuilder) อินเทอร์เฟซ ReportBuilder กำหนดเมธอดสำหรับสร้างส่วนประกอบต่าง ๆ ของรายงาน ได้แก่ buildTitle, buildTable, buildGraph, buildSummary ใช้เมธอด getResult() สำหรับดึงอ็อบเจกต์ Report ที่สร้างเสร็จแล้ว

```

1. // 2. Builder Interface: ReportBuilder
2. // กำหนดขั้นตอนการสร้างรายงาน
3. interface ReportBuilder {
4.
5.     void buildTitle(String title);
6.     void buildTable(List<List<String>> data);
7.     void buildGraph(String graphType, String graphData);
8.     void buildSummary(String summaryContent);
9.     Report getResult();
10.
11.}

```

Concrete Builders (DailyReportBuilder, MonthlyReportBuilder)

DailyReportBuilder และ MonthlyReportBuilder เป็น Builder เฉพาะสำหรับรายงานแต่ละประเภท แต่ละ Builder จะ implement อินเทอร์เฟซ ReportBuilder และให้รายละเอียดการสร้างส่วนประกอบที่แตกต่างกัน แต่ละ Builder จะเก็บออบเจกต์ Report เป็นของตัวเอง และเพิ่มส่วนประกอบต่าง ๆ เข้าไปใน Report นั้น

- DailyReportBuilder จัดรูปแบบตารางหรือกราฟในแบบหนึ่ง และมีคำแนะนำชื่อรายงานที่เฉพาะเจาะจง
- MonthlyReportBuilder จัดรูปแบบตารางหรือกราฟอีกแบบหนึ่ง ใช้คำแนะนำชื่อรายงานและข้อความสรุปที่เป็นตัวพิมพ์ใหญ่เพื่อเน้นความสำคัญของรายงานประจำเดือน

```

1. // 3. Concrete Builder 1: DailyReportBuilder
2. // สร้างรายงานประจำวัน
3. class DailyReportBuilder implements ReportBuilder {
4.
5.     private Report report;
6.
7.     public DailyReportBuilder() {
8.         this.report = new Report();
9.     }
10.
11.     @Override
12.     public void buildTitle(String title) {
13.         report.setTitle("Daily Report: " + title);
14.     }
15.
16.     @Override
17.     public void buildTable(List<List<String>> data) {
18.         StringBuilder tableContent = new StringBuilder();
19.         tableContent.append("\n--- Daily Data Table ---\n");
20.         for (List<String> row : data) {
21.             tableContent.append(String.join("\t"
22.                 , row)).append("\n");
23.         }
24.         report.addSection(tableContent.toString());
25.     }
26.

```



```

27.     @Override
28.     public void buildGraph(String graphType, String graphData)
29.     {
30.         report.addSection("\n--- Daily Trend Graph ("
31.             + graphType + ") ---\n" + graphData + "\n");
32.     }
33.
34.     @Override
35.     public void buildSummary(String summaryContent) {
36.         report.addSection("\n--- Daily Summary ---\n"
37.             + summaryContent + "\n");
38.     }
39.
40.     @Override
41.     public Report getResult() {
42.         return this.report;
43.     }
44. }

```

```

1. // 3. Concrete Builder 2: MonthlyReportBuilder
2. // สร้างรายงานประจำเดือน
3. class MonthlyReportBuilder implements ReportBuilder
4. {
5.
6.     private Report report;
7.
8.     public MonthlyReportBuilder() {
9.         this.report = new Report();
10.    }
11.
12.    @Override
13.    public void buildTitle(String title) {
14.        report.setTitle("Monthly Report: "
15.            + title.toUpperCase());
16.    }
17.
18.    @Override
19.    public void buildTable(List<List<String>> data) {
20.        StringBuilder tableContent = new StringBuilder();
21.        tableContent.append("\n=== Monthly Performance Data
22.            ===\n");
23.        for (List<String> row : data) {
24.            tableContent.append(String.join(" | "
25.                , row)).append("\n");
26.        }
27.        report.addSection(tableContent.toString());
28.    }
29.
30.    @Override
31.    public void buildGraph(String graphType, String graphData)
32.    {
33.        // รายงานรายเดือนอาจจะเน้นกราฟรวม
34.        report.addSection("\n=== Cumulative Graph ("
35.            + graphType.toUpperCase() + ") ===\n"
36.            + graphData + "\n");
37.    }

```

```

38.
39.     @Override
40.     public void buildSummary(String summaryContent) {
41.         report.addSection("\n=== MONTHLY HIGHLIGHTS ===\n"
42.             + summaryContent.toUpperCase() + "\n");
43.     }
44.
45.     @Override
46.     public Report getResult() {
47.         return this.report;
48.     }
49. }

```

Director (ReportGenerator) คลาส ReportGenerator ทำหน้าที่เป็น Director มีเมธอดที่กำหนดขั้นตอนการสร้างรายงานประเภทต่าง ๆ เช่น constructDailySales, constructMonthlyFinancial, constructBasicReport Director จะรับ ReportBuilder เข้ามาเป็นพารามิเตอร์ ทำให้สามารถทำงานร่วมกับ Builder ชนิดใดก็ได้ โดยไม่ต้องรู้รายละเอียดภายในของ Builder นั้น ๆ Director จะบอกให้ Builder สร้างส่วนประกอบตามลำดับที่กำหนดไว้สำหรับรายงานแต่ละประเภท

```

1. // 4. Director: ReportGenerator (จัดการขั้นตอนการสร้างรายงาน)
2. class ReportGenerator {
3.
4.     public void constructDailySales(ReportBuilder builder
5.         , String date) {
6.         builder.buildTitle("Sales " + date);
7.         builder.buildTable(List.of(
8.             List.of("Product", "Qty", "Revenue"),
9.             List.of("A", "100", "$1000"),
10.            List.of("B", "50", "$750")
11.        ));
12.        builder.buildGraph("Bar Chart"
13.            , "Daily Sales Performance Graph Data...");
14.        builder.buildSummary("Total daily revenue was $1750.
15.            Product A performed well.");
16.    }
17.
18.    public void constructMonthlyFinancial(
19.        ReportBuilder builder
20.        , String month) {
21.        builder.buildTitle("Financials " + month);
22.        builder.buildTable(List.of(
23.            List.of("Category", "Income", "Expense"),
24.            List.of("Sales", "$50000", "$15000"),
25.            List.of("Marketing", "$0", "$5000")
26.        ));
27.        builder.buildSummary("Monthly income exceeded expenses.
28.            Marketing spend was significant.");
29.        // รายงานรายเดือนอาจจะไม่มีกราฟ หรือมีแบบอื่น
30.    }
31. }

```

```

32.     public void constructBasicReport(
33.         ReportBuilder builder, String topic) {
34.         builder.buildTitle("Basic Report on " + topic);
35.         builder.buildSummary("This is a simple report
36.             with just a summary.");
37.     }
38.}

```

Client Code (ReportBuilderDemo) หรือ main program

- สร้างออบเจกต์ ReportGenerator (Director)
- สร้าง DailyReportBuilder หรือ MonthlyReportBuilder (Concrete Builder)
- เรียกเมธอดของ ReportGenerator เพื่อสั่งให้สร้างรายงานโดยใช้ Builder ที่เลือก
- เรียก getResult() จาก Builder เพื่อรับออบเจกต์ Report ที่สมบูรณ์มาใช้งาน

```

1. // 5. Client Code: Main application
2. public class ReportBuilderDemo {
3.     public static void main(String[] args) {
4.
5.         ReportGenerator generator = new
6.             ReportGenerator();
7.
8.         // สร้างรายงานประจำวัน
9.         System.out.println("Building Daily Report:");
10.        DailyReportBuilder dailyBuilder = new
11.            DailyReportBuilder();
12.        generator.constructDailySales(dailyBuilder
13.            , "2025-06-27");
14.        Report dailyReport = dailyBuilder.getResult();
15.        dailyReport.displayReport();
16.
17.        // สร้างรายงานประจำเดือน
18.        System.out.println("Building Monthly Report:");
19.        MonthlyReportBuilder monthlyBuilder =
20.            new MonthlyReportBuilder();
21.        generator.constructMonthlyFinancial(
22.            monthlyBuilder, "June 2025");
23.        Report monthlyReport =
24.            monthlyBuilder.getResult();
25.        monthlyReport.displayReport();
26.
27.        // สร้างรายงานพื้นฐาน
28.        // ใช้ DailyReportBuilder เพื่อแสดงความยืดหยุ่น
29.        System.out.println("Building Basic Report:");
30.        DailyReportBuilder basicBuilder = new
31.            DailyReportBuilder();
32.        generator.constructBasicReport(basicBuilder
33.            , "Project X Status");
34.        Report basicReport = basicBuilder.getResult();
35.        basicReport.displayReport();
36.
37.        // สร้างรายงานแบบกำหนดเองโดยตรงจาก Builder
38.        // ไม่ผ่าน Director

```

```

39.         System.out.println("Building Custom Report
40.                               directly:");
41.         DailyReportBuilder customBuilder = new
42.             DailyReportBuilder();
43.         customBuilder.buildTitle("Custom Ad-Hoc
44.             Analysis");
45.         customBuilder.buildGraph("Pie Chart"
46.             , "Custom Data Breakdown Graph...");
47.         customBuilder.buildSummary("Custom analysis
48.             completed successfully.");
49.         Report customReport = customBuilder.getResult();
50.         customReport.displayReport();
51.     }
52. }

```

ตัวอย่างที่ 2 การสร้างเมนูอาหาร (Menu Builder) ร้านอาหารต้องการระบบสร้างเมนูที่สามารถเลือกอาหาร เครื่องดื่ม และของหวานได้ตามต้องการ เมนูแต่ละชุดอาจมีส่วนประกอบที่แตกต่างกันสามารถใช้ Builder Pattern เพื่อสร้างเมนูอาหารแบบยืดหยุ่น

ตัวอย่างที่ 3 การสร้างเกมตัวละคร (Game Character Creation) เกม RPG ต้องการระบบสร้างตัวละครที่ผู้เล่นสามารถเลือกอาวุธ ชุดเกราะ และความสามารถพิเศษได้ตามต้องการ ตัวละครแต่ละตัวอาจมีอุปกรณ์หรือคุณสมบัติแตกต่างกันสามารถใช้ Builder Pattern เพื่อสร้างตัวละครที่มีความยืดหยุ่นในการกำหนดค่า

ข้อดีของ Builder Pattern

- แยกกระบวนการสร้างออกจากโครงสร้าง ลดความซับซ้อนของ Constructor
- รองรับอ็อบเจกต์ที่ซับซ้อน ช่วยให้สร้างอ็อบเจกต์ได้ที่ละขั้นตอน
- ยืดหยุ่นและปรับแต่งได้ สามารถกำหนดเฉพาะส่วนที่ต้องการ
- ลดการใช้พารามิเตอร์จำนวนมากใน Constructor

3.5 Prototype Pattern

Prototype Design Pattern เป็นแพตเทิร์นที่ช่วยให้สามารถสร้างอ็อบเจกต์ใหม่ด้วยการคัดลอก (clone) อ็อบเจกต์ที่มีอยู่แล้วได้โดยตรงแทนการสร้างใหม่ทั้งหมด เหมาะสำหรับสถานการณ์ที่ต้องการสร้างอ็อบเจกต์มีความซับซ้อนหรือมีค่าใช้จ่ายสูง

:: *Prototype Pattern* ::

สร้างอ็อบเจกต์ใหม่โดยคัดลอก (clone) อ็อบเจกต์ที่มีอยู่แล้ว

เหมาะสำหรับสถานการณ์การสร้างอ็อบเจกต์มีความซับซ้อน หรือมีค่าใช้จ่ายสูง

ตัวอย่างปัญหา

ต้องการสร้างบ้านหลายหลังที่มีคุณสมบัติคล้ายกัน เช่น แบบบ้านเหมือนกัน แตกต่างกันในเรื่องรายละเอียดเล็กน้อย เช่น สีของบ้าน

ตัวอย่าง Main program

```
// สร้าง house1 จาก prototypeHouse
House prototypeHouse = new House("Detached", 3, 2, "White");
House house1 = prototypeHouse.clone();
house1.setColor("Blue"); // แก้ไขเฉพาะสีบ้าน
```

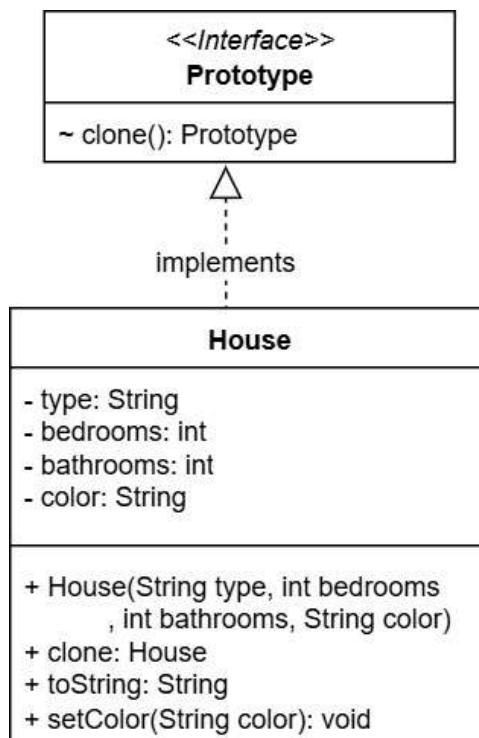
3.5.1 วัตถุประสงค์และตัวอย่าง

วัตถุประสงค์

- ลดค่าใช้จ่ายในการสร้างอ็อบเจกต์ การสร้างอ็อบเจกต์ใหม่จากศูนย์อาจใช้ทรัพยากรมาก หากสามารถคัดลอกอ็อบเจกต์เดิม และแก้ไขเฉพาะส่วนที่ต้องการ จะช่วยลดเวลา และค่าใช้จ่ายได้
- รองรับอ็อบเจกต์ที่มีค่าหรือโครงสร้างเริ่มต้น สามารถเริ่มต้นอ็อบเจกต์ใหม่จากค่าของอ็อบเจกต์ที่มีอยู่
- มีความยืดหยุ่นในการปรับแต่ง ผู้ใช้งานสามารถคัดลอกอ็อบเจกต์และแก้ไขค่าบางส่วนได้ตามต้องการ

ตัวอย่างการใช้งาน

พิจารณาสถานการณ์ที่ต้องการสร้างบ้านหลายหลังที่มีคุณสมบัติคล้ายกัน เช่น แบบบ้านเหมือนกัน แตกต่างกันในเรื่องรายละเอียดเล็กน้อย เช่น สีของบ้าน



ภาพที่ xx สร้างบ้านหลายหลังที่มีคุณสมบัติคล้ายกันด้วย Prototype pattern

Interface `Prototype` ใช้กำหนดว่าอ็อบเจกต์ที่ต้องการรองรับ `Prototype` มีเมธอด `clone()` เพื่อคัดลอกตัวเอง

```
1. // Step 1: สร้าง Interface Cloneable
2. interface Prototype extends Cloneable {
3.     Prototype clone();
4. }
```

คลาส `House` Implement Interface `Prototype` และ Override Method `clone()` เพื่อรองรับการคัดลอก มี เมธอด `setColor()` สำหรับการปรับแต่งค่าหลังจากคัดลอก

```
1. // Step 2: สร้างคลาส House ที่ใช้ Prototype
2. class House implements Prototype {
3.
4.     // Attributes
5.     private String type; // ประเภทบ้าน (เช่น บ้านเดี่ยว, ทาวน์เฮาส์)
```

```

6.     private int bedrooms;          // จำนวนห้องนอน
7.     private int bathrooms;        // จำนวนห้องน้ำ
8.     private String color;         // สีของบ้าน
9.
10.    public House(String type, int bedrooms, int bathrooms
11.                  , String color) {
12.        this.type = type;
13.        this.bedrooms = bedrooms;
14.        this.bathrooms = bathrooms;
15.        this.color = color;
16.    }
17.
18.    // Override Method clone
19.    @Override
20.    public House clone() {
21.        try {
22.            return (House) super.clone();
23.        } catch (CloneNotSupportedException e) {
24.            throw new RuntimeException(
25.                "Clone not supported for this object.");
26.        }
27.    }
28.
29.    // Method สำหรับแสดงข้อมูลบ้าน
30.    @Override
31.    public String toString() {
32.        return "House [Type: " + type + ", Bedrooms: "
33.            + bedrooms + ", Bathrooms: " + bathrooms
34.            + ", Color: " + color + "]\n";
35.    }
36.
37.    // Setter สำหรับการแก้ไขค่าบางส่วน
38.    public void setColor(String color) {
39.        this.color = color;
40.    }
41.}

```

การใช้งานใน Main สร้างบ้านต้นแบบ (Prototype) คัดลอกบ้านจาก Prototype และแก้ไขค่าบางส่วน เช่น สีบ้าน

```

1. // Step 3: การใช้งาน Prototype
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // สร้างบ้านต้นแบบ
6.         House prototypeHouse = new House("Detached"
7.             , 3, 2, "White");
8.
9.         // สร้างบ้านใหม่โดยการคัดลอกจากต้นแบบ
10.        House house1 = prototypeHouse.clone();
11.        house1.setColor("Blue"); // แก้ไขสีบ้าน
12.
13.        House house2 = prototypeHouse.clone();
14.        house2.setColor("Green"); // แก้ไขสีบ้าน
15.

```

```

16.          // แสดงข้อมูลบ้านแต่ละหลัง
17.          System.out.println("Prototype House: "
18.                               + prototypeHouse);
19.          System.out.println("House 1: " + house1);
20.          System.out.println("House 2: " + house2);
21.      }
22.  }

```

ผลลัพธ์ของโปรแกรม

Prototype House: House [Type: Detached, Bedrooms: 3, Bathrooms: 2, Color: White]

House 1: House [Type: Detached, Bedrooms: 3, Bathrooms: 2, Color: Blue]

House 2: House [Type: Detached, Bedrooms: 3, Bathrooms: 2, Color: Green]

ข้อดีของ Prototype Pattern

- ลดความซับซ้อน ไม่ต้องสร้างอ็อบเจกต์ใหม่จากศูนย์
- ประหยัดเวลาและทรัพยากร คัดลอกอ็อบเจกต์ที่มีอยู่แล้วแทนการสร้างใหม่
- ยืดหยุ่น สามารถปรับแต่งอ็อบเจกต์ที่คัดลอกได้ตามต้องการ

3.5.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Prototype Pattern เหมาะกับการใช้งานที่ต้องการสร้างอ็อบเจกต์ที่มีความซับซ้อน หรือมีคุณสมบัติคล้ายกันหลายตัว โดยการคัดลอก (clone) จากอ็อบเจกต์ต้นแบบ แทนการสร้างใหม่ทั้งหมด ตัวอย่างปัญหาในงานจริงที่สามารถแก้ไขได้ด้วย Prototype Pattern ได้แก่

ตัวอย่างที่ 1 การสร้างเอกสารทางธุรกิจ (Document Management System) ในระบบจัดการเอกสาร เช่น ใบแจ้งหนี้ ใบเสนอราคา หรือรายงาน แต่ละเอกสารมีรูปแบบที่คล้ายกัน แต่ข้อมูลภายในแตกต่างกัน การสร้างเอกสารใหม่ทุกครั้งจะเสียเวลา และมีความเสี่ยงที่จะเขียนโค้ดซ้ำซ้อนสามารถใช้ Prototype Pattern เพื่อสร้างเอกสารใหม่โดยการคัดลอก (clone) จากต้นแบบ แล้วแก้ไขเฉพาะข้อมูลที่แตกต่าง

```

1. class Document implements Cloneable {
2.
3.     // Attributes
4.     private String type;
5.     private String content;
6.
7.     public Document(String type, String content) {
8.         this.type = type;

```



```

9.         this.content = content;
10.    }
11.
12.    @Override
13.    protected Document clone() {
14.        try {
15.            return (Document) super.clone();
16.        } catch (CloneNotSupportedException e) {
17.            throw new RuntimeException("Clone not supported");
18.        }
19.    }
20.
21.    public void setContent(String content) {
22.        this.content = content;
23.    }
24.
25.    @Override
26.    public String toString() {
27.        return "Document [Type: " + type + ", Content: "
28.            + content + "]";
29.    }
30.}

```

```

1. public class Main {
2.     public static void main(String[] args) {
3.         Document invoiceTemplate = new Document("Invoice"
4.             , "Template Content");
5.
6.         Document invoice1 = invoiceTemplate.clone();
7.         invoice1.setContent("Invoice for Customer A");
8.
9.         Document invoice2 = invoiceTemplate.clone();
10.        invoice2.setContent("Invoice for Customer B");
11.
12.        System.out.println(invoiceTemplate); // ดัชนีแบบ
13.        System.out.println(invoice1); // ใบแจ้งหนี้สำหรับลูกค้า A
14.        System.out.println(invoice2); // ใบแจ้งหนี้สำหรับลูกค้า B
15.    }
16.}

```

ผลลัพธ์

Document [Type: Invoice, Content: Template Content]

Document [Type: Invoice, Content: Invoice for Customer A]

Document [Type: Invoice, Content: Invoice for Customer B]

ตัวอย่างที่ 2 การสร้างตัวละครในเกม (Game Character Creation) ในเกม RPG ผู้เล่นสามารถสร้างตัวละครที่มีคุณสมบัติพื้นฐานคล้ายกัน เช่น พลังชีวิต พลังโจมตี และชุดเกราะ แต่ตัวละครอาจมีความแตกต่างเล็กน้อย เช่น ชื่อ ทักษะเฉพาะ หรืออาวุธ หากสร้างตัวละครแต่ละตัวใหม่จากศูนย์จะ

เสียเวลา สามารถใช้ Prototype Pattern เพื่อคัดลอกตัวละครต้นแบบ แล้วปรับค่าบางส่วนตาม
ต้องการ

```
1. class GameCharacter implements Cloneable {
2.
3.     // Attributes
4.     private String name;
5.     private String weapon;
6.     private int health;
7.
8.     public GameCharacter(String name, String weapon
9.                           , int health) {
10.         this.name = name;
11.         this.weapon = weapon;
12.         this.health = health;
13.     }
14.
15.     @Override
16.     protected GameCharacter clone() {
17.         try {
18.             return (GameCharacter) super.clone();
19.         } catch (CloneNotSupportedException e) {
20.             throw new RuntimeException("Clone not supported");
21.         }
22.     }
23.     public void setName(String name) {
24.         this.name = name;
25.     }
26.
27.     @Override
28.     public String toString() {
29.         return "GameCharacter [Name: " + name + ", Weapon: "
30.             + weapon + ", Health: " + health + "]";
31.     }
32.}

1. public class Main {
2.     public static void main(String[] args) {
3.
4.         GameCharacter prototype = new GameCharacter("Default"
5.             , "Sword", 100);
6.
7.         GameCharacter player1 = prototype.clone();
8.         player1.setName("Player1");
9.
10.        GameCharacter player2 = prototype.clone();
11.        player2.setName("Player2");
12.
13.        System.out.println(prototype); // ต้นแบบ
14.        System.out.println(player1);   // ตัวละคร Player1
15.        System.out.println(player2);   // ตัวละคร Player2
16.    }
17.}
```

ผลลัพธ์

GameCharacter [Name: Default, Weapon: Sword, Health: 100]

GameCharacter [Name: Player1, Weapon: Sword, Health: 100]

GameCharacter [Name: Player2, Weapon: Sword, Health: 100]

ตัวอย่างที่ 3 การสร้างบ้านในโครงการหมู่บ้านจัดสรร (Housing Projects) ในโครงการหมู่บ้านจัดสรร บ้านแต่ละหลังอาจมีโครงสร้างเหมือนกัน เช่น จำนวนห้องนอน ห้องน้ำ หรือแบบหลังคา แต่แตกต่างกันในรายละเอียดเล็กน้อย เช่น สีของบ้าน วัสดุพื้น หรือขนาดของสวน หากสร้างบ้านแต่ละหลังใหม่จากศูนย์ จะสิ้นเปลืองเวลาและทรัพยากร สามารถใช้ Prototype Pattern เพื่อสร้างบ้านใหม่โดยการคัดลอกจากต้นแบบ แล้วปรับแต่งค่าต่าง ๆ

```
1. class House implements Cloneable {
2.
3.     // Attributes
4.     private String type;
5.     private int bedrooms;
6.     private int bathrooms;
7.     private String color;
8.
9.     public House(String type, int bedrooms, int bathrooms
10.        , String color) {
11.         this.type = type;
12.         this.bedrooms = bedrooms;
13.         this.bathrooms = bathrooms;
14.         this.color = color;
15.     }
16.
17.     @Override
18.     protected House clone() {
19.         try {
20.             return (House) super.clone();
21.         } catch (CloneNotSupportedException e) {
22.             throw new RuntimeException("Clone not supported");
23.         }
24.     }
25.
26.     public void setColor(String color) {
27.         this.color = color;
28.     }
29.
30.     @Override
31.     public String toString() {
32.         return "House [Type: " + type + ", Bedrooms: "
33.             + bedrooms + ", Bathrooms: " + bathrooms
34.             + ", Color: " + color + "]\n";
35.     }
36. }
```

```

1. public class Main {
2.     public static void main(String[] args) {
3.
4.         House prototypeHouse = new House("Detached"
5.             , 3, 2, "White");
6.
7.         House house1 = prototypeHouse.clone();
8.         house1.setColor("Blue");
9.
10.        House house2 = prototypeHouse.clone();
11.        house2.setColor("Green");
12.
13.        System.out.println(prototypeHouse); // ต้นแบบ
14.        System.out.println(house1); // บ้านหลังแรก
15.        System.out.println(house2); // บ้านหลังที่สอง
16.    }
17.}

```

ผลลัพธ์

House [Type: Detached, Bedrooms: 3, Bathrooms: 2, Color: White]

House [Type: Detached, Bedrooms: 3, Bathrooms: 2, Color: Blue]

House [Type: Detached, Bedrooms: 3, Bathrooms: 2, Color: Green]

ข้อดีของ Prototype Pattern

- ลดต้นทุนการสร้างอ็อบเจกต์ เหมาะสำหรับอ็อบเจกต์ที่มีการกำหนดค่าหรือโครงสร้างที่ซับซ้อน
- ยืดหยุ่น สามารถปรับแต่งอ็อบเจกต์ที่คัดลอกมาได้ตามต้องการ
- รวดเร็วและประหยัดทรัพยากร การคัดลอกอ็อบเจกต์ใช้ทรัพยากรน้อยกว่าการสร้างใหม่ทั้งหมด

3.6 Singleton pattern

Singleton Design Pattern มีวัตถุประสงค์เพื่อกำหนดให้คลาสหนึ่งที่ต้องการมีเพียง 1 อินสแตนซ์ (instance) เท่านั้น ในขณะที่ระบบกำลังทำงานอยู่ และสามารถเข้าถึงอินสแตนซ์หนึ่งเดียวนั้นจากคลาสอื่น ๆ ได้

:: *Singleton pattern* ::

กำหนดให้คลาสหนึ่งที่ต้องการมีเพียง 1 อินสแตนซ์ (instance) เท่านั้น ในขณะที่ระบบกำลังทำงานอยู่

สามารถเข้าถึงอินสแตนซ์หนึ่งเดียวนั้นจากคลาสอื่น ๆ ได้

ตัวอย่างปัญหา

คลาส “เซฟเฮ้าส์” (Safehouse) ซึ่งเป็นบ้านเพื่อความปลอดภัย มีเพียงหลังเดียวเท่านั้น เนื่องจากใช้เก็บอุปกรณ์ต่าง ๆ ที่จำเป็นสำหรับการรักษาความปลอดภัยของหมู่บ้าน ระบบจัดการหมู่บ้านต้องการให้ทุกโมดูล (module) ของระบบเข้าถึงข้อมูลของเซฟเฮ้าส์นี้ร่วมกันเท่านั้น

ตัวอย่าง Main program

```
// การเรียกใช้งาน instance ของ SafeHouse
SafeHouse house1 = SafeHouse.getInstance("85 บ้านศรีโค");
house1.showAddress();
```

3.6.1 วัตถุประสงค์และตัวอย่าง

Singleton pattern ช่วยหลีกเลี่ยงข้อผิดพลาดที่อาจเกิดขึ้นจากการมีอินสแตนซ์ของอ็อบเจกต์ซ้ำซ้อน ดังนี้

- จำกัดการสร้างอ็อบเจกต์ซ้ำซ้อน ลดการใช้ทรัพยากร
- ควบคุมทรัพยากรที่ใช้ร่วมกัน (Shared Resource) เช่น Connection Pool, Configuration, Logger หรือ Cache ซึ่งควรมีอินสแตนซ์เดียวที่ใช้ร่วมกัน
- ทำให้การเข้าถึงอ็อบเจกต์มีความเป็นเอกภาพ ทุกจุดของโปรแกรมที่เรียกใช้จะใช้งานอินสแตนซ์เดียวกันเสมอ

ตัวอย่างการใช้งาน

สมมติว่ามีคลาส “เซฟเฮ้าส์” (Safehouse) ซึ่งเป็นบ้านเพื่อความปลอดภัย มีเพียงหลังเดียวเท่านั้น เนื่องจากใช้เก็บอุปกรณ์ต่าง ๆ ที่จำเป็นสำหรับการรักษาความปลอดภัยของหมู่บ้าน ระบบจัดการหมู่บ้านต้องการให้ทุกโมดูล (module) ของระบบเข้าถึงข้อมูลของเซฟเฮ้าส์นี้ร่วมกันเท่านั้น เพื่อไม่ให้เกิดการสร้างข้อมูลเกี่ยวกับความปลอดภัยซ้ำซ้อน

SafeHouse
- instance: SafeHouse - address: String
- SafeHouse(String address) + getInstance(String address): SafeHouse + showAddress(): void

ภาพที่ xx คลาสเซฟเฮ้าส์มีเพียงหลังเดียวด้วย Singleton pattern

ตัวอย่างด้านล่าง ที่อยู่ของ SafeHouse คือ “85 บ้านศรีไค” เท่านั้น จะเปลี่ยนที่อยู่ไม่ได้

สร้างอินสแตนซ์เดียวของ SafeHouse

- กำหนด static SafeHouse
- สร้างอ็อบเจกต์ ทำได้เพียงครั้งเดียวผ่านเมธอด getInstance()
- ป้องกันการสร้างอ็อบเจกต์ซ้ำซ้อน โดยกำหนดให้ Constructor เป็น private

```
1. // Singleton Class
2. class SafeHouse {
3.
4.     // Static instance ตัวเดียวของคลาส SafeHouse
5.     private static SafeHouse instance;
6.
7.     // ที่อยู่ของ SafeHouse
8.     private String address;
9.
10.    // Constructor ต้องเป็น private เพื่อป้องกันการสร้างอ็อบเจกต์ภายนอก
11.    private SafeHouse(String address) {
12.        this.address = address;
13.    }
14.
15.    // Static method สำหรับการเข้าถึง instance เดียวเท่านั้น
16.    public static SafeHouse getInstance(String address) {
```

```

17.         if (instance == null) {
18.             // สร้าง instance เพียงครั้งเดียว
19.             instance = new SafeHouse(address);
20.         }
21.         return instance;
22.     }
23.
24.     // Method ตัวอย่าง
25.     public void showAddress() {
26.         System.out.println("The house address is: " + address);
27.     }
28. }

```

การใช้งาน SafeHouse ร่วมกัน (Shared Resource) คือถูกเรียกใช้ร่วมกันทั่วทั้งระบบ

```

1. // Class หลักสำหรับทดสอบ
2. public class DemoSigleton {
3.     public static void main(String[] args) {
4.
5.         // การเรียกใช้งาน instance ของ SafeHouse
6.         SafeHouse house1 = SafeHouse
7.             .getInstance("85 บ้านศรีไค");
8.         house1.showAddress();
9.
10.        // การพยายามสร้างเซฟเฮ้าส์ใหม่ แต่จะได้ instance เดิมเสมอ
11.        SafeHouse house2 = SafeHouse.getInstance("58 บ้านธาตุ");
12.        house2.showAddress();
13.
14.        // ตรวจสอบว่า house1 และ house2 เป็น instance เดียวกันหรือไม่
15.        if (house1 == house2) {
16.            System.out.println("house1 and house2
17.                are the same instance.");
18.        } else {
19.            System.out.println("Different instances exist.");
20.        }
21.    }
22. }

```

ผลลัพธ์ของโปรแกรม

The house address is: 85 บ้านศรีไค

The house address is: 85 บ้านศรีไค

house1 and house2 are the same instance.

3.6.2 ปัญหาที่เกิดจากการสร้างอ็อบเจกต์ซ้ำซ้อน

การสร้างอ็อบเจกต์ซ้ำซ้อนในกรณีที่เราจะมีเพียงแค่อินสแตนซ์ (instance) เดียว อาจนำไปสู่ปัญหาดังนี้

- ข้อมูลไม่สอดคล้องกัน (Inconsistent Data)
- สิ้นเปลืองหน่วยความจำโดยไม่จำเป็น
- เกิดข้อผิดพลาดในการประมวลผลข้อมูล (Race Condition)
- กระบวนการควบคุมทรัพยากรทำงานผิดพลาด

ตัวอย่างข้อผิดพลาดเมื่อไม่ใช่ Singleton Pattern

```
1. // Class SafeHouse ที่ไม่ใช่ Singleton Design
2. class SafeHouse {
3.
4.     private String address;
5.
6.     public SafeHouse(String address) {
7.         this.address = address;
8.     }
9.     public void showAddress() {
10.        System.out.println("The house address is: " + address);
11.    }
12.}
```

```
1. // Class หลัก - สร้างอ็อบเจกต์ซ้ำซ้อน
2. public class DemoSingleton {
3.     public static void main(String[] args) {
4.
5.         // สร้าง อ็อบเจกต์ ของ SafeHouse หลายครั้ง
6.         SafeHouse house1 = new SafeHouse("85 บ้านศรีโค");
7.         SafeHouse house2 = new SafeHouse("58 บ้านธาตุ");
8.
9.         // แสดงผลลัพธ์
10.        house1.showAddress();
11.        house2.showAddress();
12.
13.        // ข้อผิดพลาด: instance ไม่ใช่ตัวเดียวกัน
14.        if (house1 != house2) {
15.            System.out.println("ERROR: Two separate
16.                                instances of House exist!");
17.        }
18.    }
19.}
```


ผลลัพธ์ของโปรแกรม

The house address is: 85 บ้านศรีไค

The house address is: 58 บ้านธาตุ

ERROR: Two separate instances of House exist!

คำอธิบายข้อผิดพลาด

- ข้อมูลไม่สอดคล้องกัน (Inconsistent Data) SafeHouse ควรจะมีหลังเดียว แต่มีการสร้างอ็อบเจกต์ house1 และ house2 ขึ้นมา ทำให้มีบ้านสองหลังที่มีที่อยู่ต่างกัน
house1 มีที่อยู่เป็น 85 บ้านศรีไค
house2 มีที่อยู่เป็น 58 บ้านธาตุ
- ข้อมูลไม่ตรงกันและทำให้ระบบไม่สามารถรู้ได้ว่าที่อยู่ไหนเป็นที่อยู่ “จริง” ของ SafeHouse สลับเปลี่ยนหน่วยความจำโดยไม่จำเป็น การสร้างอ็อบเจกต์ หลาย ๆ ตัว โดยไม่จำเป็น ทำให้หน่วยความจำถูกใช้งานเกินความจำเป็น
- ความผิดพลาดเชิงตรรกะ (Logic Errors) ถ้าระบบต้องการใช้งานเซฟเฮ้าส์หลังเดียวเป็นศูนย์กลาง เช่น ระบบควบคุมประตูหน้าหมู่บ้าน หรือการเชื่อมต่อกับระบบอื่น แต่กลับมีหลาย instance ของเซฟเฮ้าส์ ทำให้เกิดข้อมูลผิดพลาดได้ง่าย

3.6.3 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

การใช้ Singleton Pattern ในสถานการณ์การทำงานจริงมีความจำเป็นเมื่อโปรแกรมหรือระบบต้องการให้อ็อบเจกต์ 1 ตัวเท่านั้นเป็นตัวแทนในการควบคุม หรือจัดการกระบวนการที่สำคัญ เพื่อป้องกันปัญหาการซ้ำซ้อน และความไม่สอดคล้องกัน ตัวอย่างปัญหาที่จำเป็นต้องใช้ Singleton Pattern ได้แก่

ตัวอย่างที่ 1 การจัดการข้อมูลการตั้งค่า (Configuration) ระบบซอฟต์แวร์มักมีการตั้งค่าหรือค่าคอนฟิก (configuration) ที่ใช้ร่วมกัน เช่น การเชื่อมต่อกับฐานข้อมูล, URL ของ API, หรือพารามิเตอร์ระบบ หากมีการสร้างอ็อบเจกต์ซ้ำหลายตัว ข้อมูลการตั้งค่าที่ใช้ในแต่ละส่วนของโปรแกรมอาจไม่ตรงกันหรือเกิดปัญหาความขัดแย้งของข้อมูล ควรใช้ Singleton Pattern เพื่อให้แน่ใจว่าข้อมูลการตั้งค่าถูกจัดเก็บและเข้าถึงผ่านอ็อบเจกต์เดียวเท่านั้น

```
1. class ConfigurationManager {  
2.  
3.     private static ConfigurationManager instance;  
4.     private String config;  
5.
```

```

6.     private ConfigurationManager() {
7.         // Load configuration from file or database
8.         config =
9.             "DatabaseURL=jdbc:mysql://localhost:3306/mydb";
10.    }
11.
12.    public static ConfigurationManager getInstance() {
13.        if (instance == null) {
14.            instance = new ConfigurationManager();
15.        }
16.        return instance;
17.    }
18.
19.    public String getConfig() {
20.        return config;
21.    }
22.}

```

```

1. // การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         ConfigurationManager config1 =
6.             ConfigurationManager.getInstance();
7.         ConfigurationManager config2 =
8.             ConfigurationManager.getInstance();
9.
10.        // Output: DatabaseURL=jdbc:mysql://localhost:3306/mydb
11.        System.out.println(config1.getConfig());
12.        // Output: true (เป็นอ็อบเจกต์ เดียวกัน)
13.        System.out.println(config1 == config2);
14.    }
15.}

```

ตัวอย่างที่ 2 การจัดการ Logging ในระบบที่ซับซ้อนจำเป็นต้องบันทึกข้อมูลการทำงาน หรือข้อผิดพลาด (Logging) หากมี Logger หลายตัว อาจทำให้ข้อมูลซ้ำซ้อนหรือไม่สอดคล้องกันในไฟล์ หรือฐานข้อมูล การใช้ Singleton Pattern ทำให้มั่นใจว่าทุกการบันทึกข้อมูลมาจาก Logger ตัวเดียวกัน

```

1. class Logger {
2.     private static Logger instance;
3.
4.     private Logger() {}
5.
6.     public static Logger getInstance() {
7.         if (instance == null) {
8.             instance = new Logger();
9.         }
10.        return instance;
11.    }
12.
13.    public void log(String message) {
14.        System.out.println("[LOG] " + message);

```

```

15.    }
16.}

```

```

1. // การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         Logger logger1 = Logger.getInstance();
6.         Logger logger2 = Logger.getInstance();
7.
8.         // Output: [LOG] This is a log message.
9.         logger1.log("This is a log message.");
10.        // Output: true (เป็นอ็อบเจกต์ เดียวกัน)
11.        System.out.println(logger1 == logger2);
12.    }
13.}

```

ตัวอย่างที่ 3 การจัดการ Database Connection Pool ระบบที่ต้องเชื่อมต่อ (Connection) กับฐานข้อมูลหลายครั้ง เช่น Web Application หากสร้าง Connection ใหม่ทุกครั้งทำให้สิ้นเปลืองทรัพยากร และล่าช้า สามารถใช้ Singleton Pattern เพื่อสร้าง Connection Pool ที่มีอ็อบเจกต์เดียวสำหรับจัดการการเชื่อมต่อกับฐานข้อมูล

```

1. class DatabaseConnection {
2.
3.     private static DatabaseConnection instance;
4.
5.     private DatabaseConnection() {
6.         System.out.println("Database connection created.");
7.     }
8.
9.     public static DatabaseConnection getInstance() {
10.        if (instance == null) {
11.            instance = new DatabaseConnection();
12.        }
13.        return instance;
14.    }
15.
16.    public void query(String sql) {
17.        System.out.println("Executing query: " + sql);
18.    }
19.}

```

```

1. // การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         DatabaseConnection connection1 =
6.             DatabaseConnection.getInstance();
7.         DatabaseConnection connection2 =
8.             DatabaseConnection.getInstance();
9.

```

```

10.     connection1.query("SELECT * FROM users;");
11.     connection2.query("UPDATE users SET name = 'John';");
12.     // Output: true (เป็นอ็อบเจกต์ เดียวกัน)
13.     System.out.println(connection1 == connection2);
14. }
15.}

```

ตัวอย่างที่ 4 การจัดการ Cache ระบบที่ต้องเก็บข้อมูลที่ถูกรับมาใช้งานบ่อย เช่น รายการสินค้าหรือผู้ใช้งาน หากสร้าง Cache ของหลายอ็อบเจกต์ จะทำให้ข้อมูลกระจาย และไม่สามารถควบคุมได้ ดังนั้นจึงควรใช้ Singleton Pattern เพื่อให้ Cache มีอ็อบเจกต์เดียวที่ใช้ร่วมกันในโปรแกรม

```

1. class Cache {
2.
3.     private static Cache instance;
4.     private Map<String, String> data;
5.
6.     private Cache() {
7.         data = new HashMap<>();
8.     }
9.
10.    public static Cache getInstance() {
11.        if (instance == null) {
12.            instance = new Cache();
13.        }
14.        return instance;
15.    }
16.
17.    public void put(String key, String value) {
18.        data.put(key, value);
19.    }
20.
21.    public String get(String key) {
22.        return data.get(key);
23.    }
24.}

```

```

1. // การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         Cache cache1 = Cache.getInstance();
6.         Cache cache2 = Cache.getInstance();
7.
8.         cache1.put("user1", "Alice");
9.         // Output: Alice
10.        System.out.println(cache2.get("user1"));
11.        // Output: true (เป็นอ็อบเจกต์ เดียวกัน)
12.        System.out.println(cache1 == cache2);
13.    }
14.}

```

3.7 บทสรุป

แพตเทิร์นการสร้างวัตถุ (Creational Patterns) เป็นหนึ่งในสามประเภทหลักของ Design Patterns (Creational, Structural, Behavioral) ซึ่งได้รับการแนะนำครั้งแรกในหนังสือ “Design Patterns: Elements of Reusable Object-Oriented Software” โดย Gang of Four (GoF) ในปี 1994 แพตเทิร์นกลุ่มนี้มุ่งเน้นไปที่การแก้ปัญหาการสร้างออบเจกต์ (Object Creation) ในระบบที่ซับซ้อน โดยลดความซับซ้อนของโค้ด เพิ่มความยืดหยุ่น และช่วยให้การจัดการวัตถุทำได้มีประสิทธิภาพ มีวัตถุประสงค์หลักคือจัดการการสร้างวัตถุในสถานการณ์ที่ซับซ้อน ลดการผูกมัดระหว่างคลาส ควบคุมกระบวนการสร้างวัตถุ และเพิ่มความสามารถในการนำโค้ดกลับมาใช้ใหม่

แนวคิดหลักของแพตเทิร์นการสร้างวัตถุ ได้แก่ การแยกส่วนการสร้างวัตถุ (Encapsulation of Object Creation) เป็นการซ่อนรายละเอียดการสร้างออบเจกต์จากผู้ใช้งาน เพื่อให้สามารถปรับเปลี่ยนกระบวนการสร้างได้โดยไม่กระทบต่อโค้ดที่เรียกใช้ และการลดการพึ่งพาโครงสร้างของคลาส โดยสร้างวัตถุโดยไม่ต้องพึ่งพาโครงสร้างภายในของคลาสโดยตรง เช่น การสร้างผ่าน Factory หรือ Builder

แพตเทิร์นการสร้างวัตถุ ได้แก่ Factory Method, Abstract Factory, Builder, Prototype และ Singleton มีคุณสมบัติเด่น เช่น ความยืดหยุ่น (Flexibility) สามารถเปลี่ยนกระบวนการสร้างออบเจกต์ได้โดยไม่กระทบต่อโค้ดส่วนอื่น การเพิ่มประสิทธิภาพ (Efficiency) ลดการใช้ทรัพยากร เช่น การใช้ Prototype หรือ Object Pool การส่งเสริมการใช้หลักการ OOP ได้แก่ Encapsulation, Inheritance และ Polymorphism รวมทั้งรองรับการขยายระบบ (Scalability) คือเพิ่มความสามารถในการรองรับวัตถุหรือพฤติกรรมใหม่ได้ง่าย

3.8 คำถามท้ายบท

- 1) ทำไมการควบคุมกระบวนการสร้างออบเจกต์จึงมีความสำคัญในระบบซอฟต์แวร์ที่ซับซ้อน
- 2) แพตเทิร์นการสร้างวัตถุช่วยลดความซับซ้อนของโค้ดได้อย่างไร
- 3) แพตเทิร์นการสร้างวัตถุ (Creational Patterns) คืออะไร และมีวัตถุประสงค์อย่างไร
- 4) ในสถานการณ์ใดที่ควรพิจารณาใช้แพตเทิร์นการสร้างวัตถุ เช่น Factory Method แทนการสร้างออบเจกต์ด้วยคำสั่ง new
- 5) อธิบายความแตกต่างระหว่าง Factory Method และ Abstract Factory
- 6) Builder Pattern เหมาะกับการแก้ปัญหาประเภทใด จงยกตัวอย่าง
- 7) Prototype Pattern มีข้อดีอย่างไรเมื่อเทียบกับการสร้างออบเจกต์ใหม่
- 8) Singleton Pattern ใช้ในกรณีใด และมีหลักการทำงานอย่างไร